HOLO4D: Holistic 4D Reconstruction as Geometric Control for Video Diffusion

Yushi Lan[1] **Zeren Jiang**[1] **Koichi Namekata**[1]
Xingang Pan[2] **Chuanxia Zheng**[2] **Andrea Vedaldi**[1]

[1]University of Oxford [2]Nanyang Technological University

Abstract

Reconstructing a dynamic 4D scene along a novel camera trajectory requires both accurate geometry from the source video and generative completion of disoccluded target-view regions. Feed-forward 4D reconstructors recover increasingly comprehensive input-view geometry, including depth, point maps, and dense 3D tracks, but remain tied to the observed camera stream. Camera-controlled video diffusion models (VDMs) provide strong generative priors for novel-view synthesis, yet their camera control is typically not precise enough for metric novel-view reconstruction. We argue that comprehensive input-view 4D reconstruction is an effective geometric interface between these two capabilities: properly exposed to a VDM, it turns reconstruction outputs into camera-accurate novel-view RGB videos that remain geometrically consistent under downstream 4D reconstruction. We introduce HOLO4D, a geometry-aware video-to-video framework that controls a pretrained VDM with a hybrid 4D geometric cache built from the source video. A small set of source views covering the camera motion forms the static cache, while dynamic regions are rendered from the matching source frame. The resulting hybrid RGB-D representation is combined with explicit valid masks, target-camera ray embeddings, and dense 3D tracking tokens. Under a common downstream 4D reconstructor, our generated videos yield target-view depth more consistent with the held-out reconstruction than that of the baselines, and an additional feature-level readout shows that intermediate VAE features already encode target-view depth. Experiments and ablations further show that progressively richer 3D/4D reconstruction signals improve camera adherence.

1 Introduction

Rendering a monocular video from a new viewpoint requires one to reconstruct the 4D geometry in the input video as well as to complete it generatively. As one renders the scene from a new user-specified camera trajectory, one must reflect faithfully the 3D geometry and motion observed in the source video, while also filling in regions that become disoccluded. Recent feed-forward 3D and 4D reconstructors have made rapid progress on the problem of recovering 3D geometry and motion, predicting depth, point maps, scene flow, and 3D tracks in a single forward pass [1–4]. However, these models are primarily designed to reconstruct the observed parts of the scene. Their outputs thus remain tied to the input view, as they lack the ability to complete disoccluded regions.

Camera-controlled video diffusion models (VDMs) provide the complementary generative capability. Pretrained VDMs learn strong appearance and motion priors from large-scale video data [5] and are capable of synthesizing novel views when combined with camera-control mechanisms. Existing camera-control mechanisms can be broadly categorized into two classes: implicit geometric conditioning, which injects pose, point trajectories, or ray embeddings directly into the VDM [6], and explicit

Preprint.

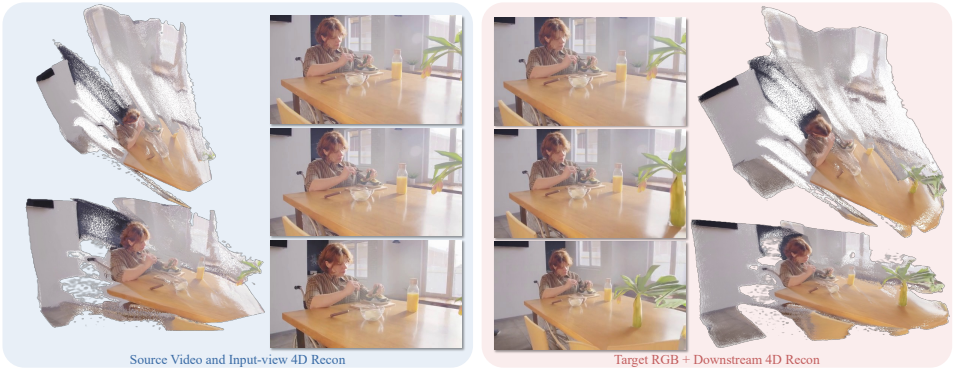


Figure 1: **Examples of camera-accurate novel-view video.** HOLO4D turns source-view 4D reconstruction into a geometric control interface for video diffusion. From a monocular source video, we recover geometry and dense 3D tracks, build a hybrid target-view condition from camera-coverage static fusion and dynamic per-frame fallback, and use it to control a pretrained VDM along a user-specified camera trajectory. The primary output is a target-view RGB video; the point cloud shown is obtained by running an off-the-shelf 4D reconstructor on this generated video.

geometric conditioning, which starts by reprojecting information into the target views (in pixel or latent space) based on the reconstructed geometry and then passes the result to the VDM [7–10]. These methods have some limitations: implicit conditioning provides a weaker geometric signal to the VDM, which tends to drift under large viewpoint changes, while explicit conditioning is often limited to reprojecting only one frame at a time, thus failing to capture the information contained in the input video as a whole, particularly in dynamic scenes.

We argue that leveraging more complete 4D reconstructions of the input video is key to improving the geometric scaffold in camera-controlled VDMs. As noted, even methods that use explicit conditioning often only use the information recovered from a single frame in the source video to generate the corresponding novel viewpoint, even though other source frames may contain useful information about disocclusions, especially for static regions. We thus consider using more structured 4D information in the VDM: fusing information from multiple source views for static regions and further injecting dense 3D tracks as motion cues for dynamic ones. In this way, feed-forward 4D reconstruction recovers depth, point maps, and persistent tracks that tell the VDM how the source video should appear from a target camera. In turn, this allows the VDM to synthesize new views more accurately, from which 4D reconstruction can recover the geometry of the novel views.

To test these ideas, we introduce HOLO4D, a geometry-aware video-to-video framework for monocular novel-view 4D reconstruction. Given a source video, estimated source geometry, dense 3D tracks, and a target camera trajectory, HOLO4D constructs target-aligned conditioning signals. The core explicit condition is a hybrid 4D geometric cache: for static scene regions, HOLO4D selects a small set of source views that cover the camera motion and fuses their point clouds before rendering them to the target trajectory; for moving regions, HOLO4D does not fuse across time and instead renders the matching source frame so that object motion stays aligned. In parallel, dense 3D tracking tokens are injected into the VDM as additional motion cues. Anchored on this hybrid 4D scaffold, the diffusion model generates target-view RGB. Beyond the RGB output, when a common off-the-shelf 4D reconstructor [11, 3, 2] is applied to every method's generated video, our outputs yield target-view depth more consistent with the held-out reconstruction than those of the baselines. We accordingly ask what geometric scaffold should be exposed to a VDM in addition to camera rays [12], and show that progressively richer 4D scaffolds improve camera adherence beyond what implicit camera control alone can achieve, as verified in Section 4.

Our contributions thus are **(1)** a **hybrid 4D geometric cache** that fuses static content across source views covering the camera motion and renders dynamic content from the matching source frame, with explicit validity masks; **(2) dense 3D tracking** as a parallel persistent-motion branch alongside the rendered RGB-D condition; and **(3) geometry-aware target-view video generation**, validated on held-out real iPhone multi-view captures, where downstream 4D reconstruction of HOLO4D-generated videos yields target-view depth more consistent with the held-out reconstruction.

2 Related Work

3D and 4D Reconstruction. Many methods for 4D reconstruction of monocular videos build on deformable neural radiance fields [13–16] or 3D Gaussians [17, 18]. For instance, Shape-of-Motion [19] parameterise the motion of the scene using a compact set of $\mathbb{SE}(3)$ trajectories which, in turn, drive 3D Gaussians. These methods reconstruct well the parts of the scene that are visible, but require costly per-scene optimization and do not extrapolate well.

Faster and more robust 3D reconstruction can be obtained from feed-forward models [20, 1, 21–28] which can recover depth, point maps, and camera motion in a single application of a neural network. Already video depth predictors can extract information useful to generate new views [29–31]. More recent approaches [32, 33, 4, 34–38, 3] can also reconstruct the dynamic parts of the scene, but only if they are visible in the input video. Here, we build on these methods to control a VDM and use the latter to complete new-views generatively.

Camera-conditioned Video Generation and Re-rendering. To apply a VDM to novel-view video generation, one must encode the target viewpoint. One way to do so is to pass to the VDM the target camera parameters, or an equivalent representation like a Plücker ray map [39–44]. However, this can sometimes lead to imprecise results, especially for large viewpoint changes, as the VDM may ignore the camera signal and follow its learned motion priors instead [42, 43].

One way to encourage the VDM to follow the camera more strictly is to first reconstruct and reproject the input geometry on the new views, and use the latter as a control signal for the VDM. MVSplat360 [7], TrajectoryCrafter [45], Gen3C [46] and others [47] do so by estimating video depth or target-view geometry, and EX-4D [48] further account for occlusions in the reprojection process. The concurrent work Vista4D [49] also reprojects depth, trying to fuse the static parts of the scene from multiple frames, and conditions the VDM on the resulting renders, dynamic region masks, and input video. Go-with-the-Track [50] also reprojects reconstructed 3D geometry or dynamic point-tracks along a target trajectory for camera control, but conditions on a sparse set of reference images anchored by point-tracks rather than a full source video. LaVR [51] conditions generation on the latent state of a large 4D reconstruction model instead of the 3D point cloud directly.

HOLO4D differs from these works in how geometry is used and what is output. First, static points are fused from a small set of source views relevant to each generated target view. For robustness, the dynamic 3D points are instead represented by encoding the corresponding dense 3D tracks as separate VDM tokens. We show that this results in new-views with a better geometric accuracy, which we test by reconstructing the corresponding 3D geometry using an off-the-shelf 4D reconstructor and comparing it to the held-out ground truth, as well as by learning a readout head that maps the VAE decoder features directly into depth, thus generating the latter as well as RGB.

Video Diffusion Models for Geometry Prediction. Pretrained generators like VDM have been used before to predict geometry, including monocular depth prediction [52, 53], or used by themselves or in combination with feed-forward reconstruction to generate jointly RGB and 3D quantities like point maps [54–58]. HOLO4D differs in inputs and outputs. Instead of tasking the VDM with inferring 4D geometry by itself, we first reconstruct the latter with an off-the-shelf model and task the VDM with completing the information missing from a new viewpoint. We then recover geometry from the generated RGB frames, or directly by learning a new readout head.

3 Method

We present HOLO4D, a geometry-aware video-to-video framework for monocular novel-view 4D synthesis. The pipeline has two stages: *(i)* build a hybrid target-view geometry condition by reprojecting source 4D reconstruction outputs into the target camera, and *(ii)* feed this condition—together with target-camera rays and dense 3D tracks—to a pretrained video diffusion backbone that synthesizes the target-view RGB. We make each stage concrete in the rest of the section: Section 3.1 fixes notation, Section 3.2 describes the hybrid 4D cache, and Section 3.3 the VDM conditioning.

3.1 Problem Setup and 4D Inputs

Given a monocular source video $\mathcal{V}^{\mathrm{s}} = \{I_i^{\mathrm{s}}\}_{i=0}^{N-1}$ captured by source cameras $\Pi^{\mathrm{s}} = \{\pi_i^{\mathrm{s}}\}_{i=0}^{N-1}$, our goal is to generate a target-view RGB video $\mathcal{V}^{\mathrm{t}}$ from a user-specified target trajectory $\Pi^{\mathrm{t}} = \{\pi_j^{\mathrm{t}}\}_{j=0}^{N-1}$.

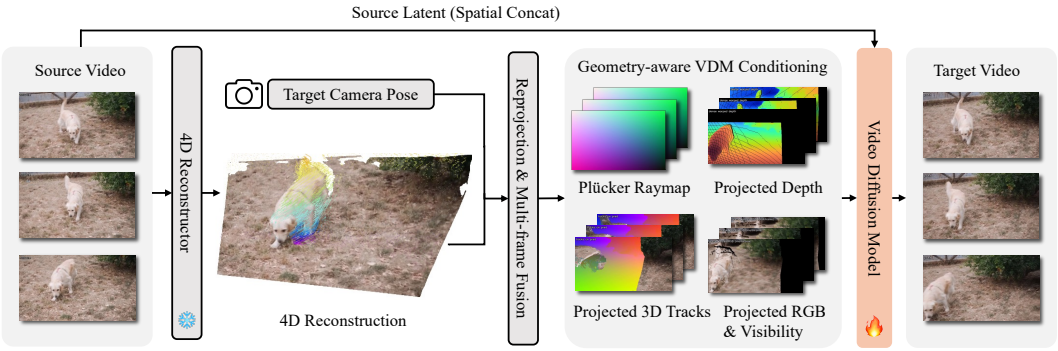


Figure 2: **HOLO4D pipeline.** An off-the-shelf 4D reconstructor estimates source geometry and 3D tracks from the input video. We split source pixels into static and dynamic regions using track motion. Static regions are fused from camera-coverage keyframes, dynamic regions use time-synchronized per-frame reprojection, and valid masks indicate observed target-view support. The resulting hybrid RGB-D condition is injected together with target-camera Plücker rays and 3D tracking tokens, and the VDM synthesizes the target-view RGB video.

At both training and test time, source camera intrinsics, extrinsics, depths $\mathcal{D}^{\mathrm{s}} = \{D_i^{\mathrm{s}}\}_{i=0}^{N-1}$, and source-view 3D tracks are produced by any off-the-shelf feed-forward 4D reconstructor (e.g., V-DPM [11] or 4RC [2]); we only fall back to ground-truth source geometry when training datasets provide it. HOLO4D itself does not produce these signals; the source point cloud and 3D tracks come from the external 4D reconstructor, and ground-truth geometry is not required at inference. Following the notation of DPM [11], $P_i(t_j)(u) \in \mathbb{R}^3$ is the 3D location at time t_j of the surface point at pixel u in source frame i, in the canonical source frame. The 'diagonal' ($i = j$) entries $P_i(t_i)$ give the visible source point maps, and off-diagonal entries give 3D tracks across time.

3.2 Hybrid 4D Geometric Cache

Most current video re-rendering methods reproject source frame j only to target frame j. This preserves temporal alignment but wastes useful static-scene observations from other source frames. HOLO4D instead builds a hybrid cache: static regions are fused from *all* source views that cover the camera motion, while moving regions are rendered from the matching source frame. We currently fuse only static regions and do not fuse the dynamic foreground across frames. Cross-time information about dynamic regions is instead carried by the dense 3D tracking tokens (Section 3.3), which encode persistent motion explicitly without requiring fused dynamic geometry.

Static-dynamic split. We identify dynamic source pixels by thresholding 3D track motion. For each pixel u in source frame i, we compute

$$m_i(u) = \max_j \left\| P_i(t_j)(u) - P_i(t_i)(u) \right\|_2, \qquad M_i^{\mathrm{dyn}}(u) = \mathbf{1}[m_i(u) > \tau_{\mathrm{dyn}}], \tag{1}$$

and set $M_i^{\mathrm{st}}(u) = 1 - M_i^{\mathrm{dyn}}(u)$. Static pixels are eligible for multi-frame fusion; dynamic pixels are excluded from the fused cache.

Camera-coverage static cache. Using all N source frames for the static cache is unnecessarily expensive and introduces redundant source views. We therefore construct a compact static source cache with $K \ll N$ camera-coverage keyframes, selected deterministically by Farthest-Point Sampling (FPS) in a normalized camera descriptor space that concatenates source-camera translation and viewing direction; this favors views that span distinct camera positions or viewing directions, especially under rotations and orbiting trajectories. The exact descriptor weighting and the value of K used in our experiments are reported in Appendix A. We lift the static pixels of the selected keyframe set $\mathcal{S}$ into a colored point cloud $\mathcal{Q}^{\mathrm{st}} = \{(P_i(t_i)(u),\, I_i^{\mathrm{s}}(u)) : i \in \mathcal{S},\ M_i^{\mathrm{st}}(u) = 1\}$. Rendering $\mathcal{Q}^{\mathrm{st}}$ from target camera π_j^{t} gives a static RGB-D render and validity mask $(\bar{I}_{j,\mathrm{st}}^{\mathrm{t}}, \bar{D}_{j,\mathrm{st}}^{\mathrm{t}}, \bar{O}_{j,\mathrm{st}}^{\mathrm{t}})$. Here $\bar{I}_{j,\mathrm{st}}^{\mathrm{t}}$ is the rendered RGB image, $\bar{D}_{j,\mathrm{st}}^{\mathrm{t}}(u)$ is the target-camera z-depth of the nearest projected point at pixel u (z-buffering resolves collisions), and $\bar{O}_{j,\mathrm{st}}^{\mathrm{t}}(u) \in \{0, 1\}$ is an occupancy mask that equals 1 iff at least one cached point projects onto u; pixels with $\bar{O}_{j,\mathrm{st}}^{\mathrm{t}}(u) = 0$ carry no geometric evidence and are left to the generative prior.

Dynamic per-frame fallback. For dynamic pixels, we do not fuse across time. At target time t_j, we render only the dynamic pixels of the synchronized source frame, $\mathcal{Q}_j^{\mathrm{dyn}} = \Big\{\big(P_j(t_j)(u),\, I_j^{\mathrm{s}}(u)\big) : M_j^{\mathrm{dyn}}(u) = 1\Big\}$, which yields the dynamic render $(\bar{I}_{j,\mathrm{dyn}}^{\mathrm{t}}, \bar{D}_{j,\mathrm{dyn}}^{\mathrm{t}}, \bar{O}_{j,\mathrm{dyn}}^{\mathrm{t}})$. We merge the two renders into $(\bar{I}_j^{\mathrm{t}}, \bar{D}_j^{\mathrm{t}}, \bar{O}_j^{\mathrm{t}})$ by per-pixel z-buffer selection: each pixel takes its content from the valid stream with smaller depth, and $\bar{O}_j^{\mathrm{t}} = \bar{O}_{j,\mathrm{st}}^{\mathrm{t}} \vee \bar{O}_{j,\mathrm{dyn}}^{\mathrm{t}}$ (formal rule in Appendix A); the hybrid target condition is then

$$\mathbf{C}_j^{\mathrm{t}} = \big[\bar{I}_j^{\mathrm{t}};\ \bar{D}_j^{\mathrm{t}};\ \bar{O}_j^{\mathrm{t}}\big] \in \mathbb{R}^{5\times H\times W} \quad (3 \text{ RGB} + 1 \text{ depth} + 1 \text{ valid mask}). \tag{2}$$

The depth channel fed to the VDM is the target-camera z-depth produced by reprojection, masked by the valid target-view support map and normalized following Geo4D [54]. The validity mask $\bar{O}_j^{\mathrm{t}}$ tells the VDM which pixels are supported by source geometry and which regions must be completed by the generative prior. Our formulation potentially supports fusing the dynamic foreground across frames as well, which we leave for future investigation.

3.3 Geometry-aware VDM Conditioning

HOLO4D conditions the VDM on four signals: the (clean) source video latent, the hybrid RGB-D cache, target-camera rays, and source-view 3D tracks. The source and noised target video latents are concatenated along the spatial width dimension [59], allowing the DiT's native attention to perform video-to-video conditioning. The hybrid RGB-D tensors $\{\mathbf{C}_j^{\mathrm{t}}\}$ are encoded by a dedicated control patch-embedding (a Conv3d) and added to the patchified video tokens at the DiT input; pixels without valid reprojection support are zero-padded, and the valid mask channel signals their location to the model. When an entire conditioning signal is unavailable, the corresponding condition adapter receives a learned null token in place of its usual input. For noise-augmented training runs, we additionally perturb geometry tokens to improve robustness to depth and splatting artifacts. Target-camera rays provide the implicit camera-control signal. For target frame j and pixel u, we compute the Plücker ray

$$\mathbf{r}_j(u) = \big(\mathbf{d}_j(u), \mathbf{o}_j \times \mathbf{d}_j(u)\big)\,, \tag{3}$$

where $\mathbf{o}_j$ is the target camera center and $\mathbf{d}_j(u)$ the ray direction, both expressed in the canonical source frame. A lightweight camera adapter maps these ray maps to DiT-resolution tokens, which are added to the self-attention input of every DiT block [6].

Finally, 3D tracks are injected as persistent motion cues rather than only being used to build the rendered cache. Each track observation at target time t_j is described by its normalized time index, its world-space position $P_i(t_j)$, a per-track visibility flag $v_i(t_j) \in \{0, 1\}$, and the depth and RGB color of the track in its source frame. The visibility flag is the valid mask of the track branch: it combines geometric visibility in the target view with the validity estimate provided by the dataset or upstream tracker, and invisible observations are zeroed out so that they contribute no signal. Each observation is encoded by a sinusoidal positional encoding followed by a shared MLP, and the resulting features are splatted onto the target-view token grid at the track's projected location; collisions are resolved by a channel-wise maximum, and cells that receive no track are filled with a learned null token. A Conv3d patchifier then produces DiT-resolution track tokens, which are injected at every DiT block by concatenating them with the video tokens and projecting back to the model dimension, in parallel with the rendered RGB-D condition.

Let $x_1 = E_{\mathrm{vae}}(\mathcal{V}^{\mathrm{t}})$ be the clean target RGB latent and let $c = \big(E_{\mathrm{vae}}(\mathcal{V}^{\mathrm{s}}), \{\mathbf{C}_j^{\mathrm{t}}\}, \{\mathbf{r}_j\}, \tau\big)$ collect the source-video, hybrid-cache, camera-ray, and track conditions. We train the RGB diffusion backbone with the standard conditional rectified-flow objective

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{x_0, x_1, c, c_{\mathrm{text}}, t}\left[\big\|u_{\boldsymbol{\theta}}(x_t, c, c_{\mathrm{text}}, t) - v_t\big\|_2^2\right], \tag{4}$$

where x_0 is a noise sample, $x_t = tx_1 + (1-t)x_0$ is the noised latent at time t, and $v_t = x_1 - x_0$ is the denoising velocity [60].

4 Experiments

4.1 Settings

Datasets. We train HOLO4D on a mixture of dynamic and static datasets spanning real and synthetic imagery, monocular and multi-camera capture, and both ground-truth and estimated geometry.

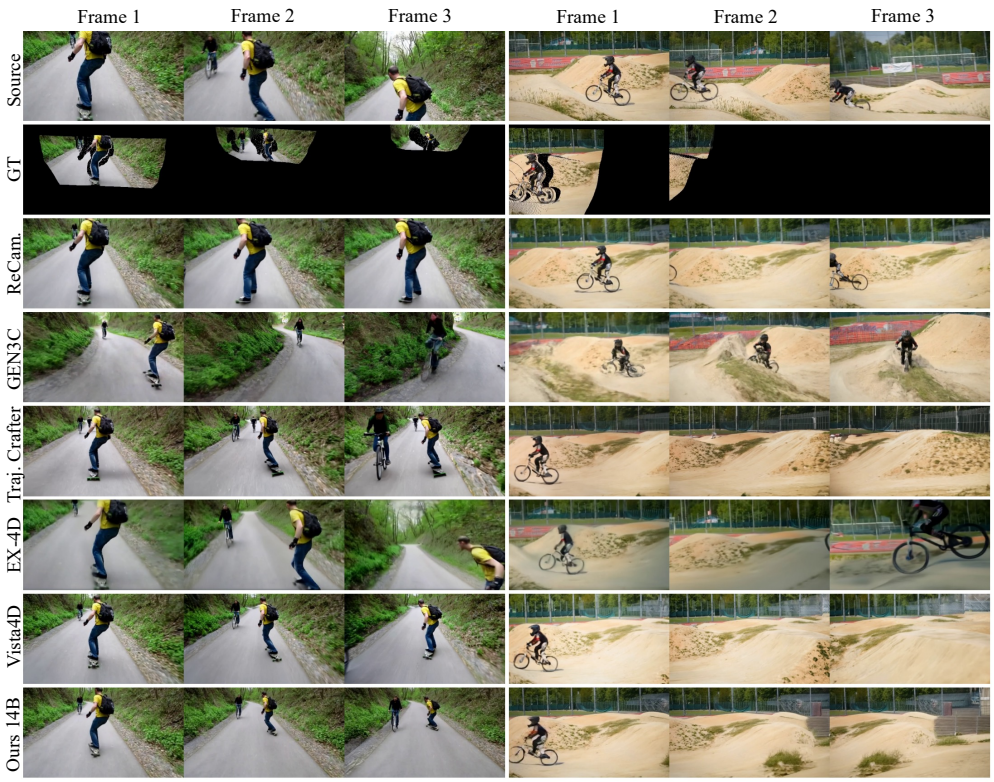


Figure 3: **Qualitative comparison on DAVIS-style real-world videos.** For each example we show the source frame, the target trajectory, and the target-view RGB synthesized by HOLO4D side-by-side with video-to-video baselines. HOLO4D follows the requested target trajectory and preserves source content under large viewpoint changes; the corresponding camera-adherence and video-quality numbers are reported in Table 1.

Dynamic training signal comes from Syn4D [61] and Kubric [62], together with the multi-camera dynamic datasets MultiCamVideo [42] and SynCamVideo [63], used here in camera-only mode. Static scenes come from DL3DV [64], RE10K [65], MVS-Synth [66], and TartanAir [67], with estimated source geometry from DepthAnything 3 [68] for real captures and ground-truth geometry for synthetic ones. Depth and 3D-track supervision are masked on a per-sample basis, so datasets without target-view depth or track annotations still contribute to RGB generation and camera-control training. Quantitative camera-adherence and video-quality evaluation uses DAVIS [69] (Appendix D); novel-view RGB and downstream-reconstruction depth on real captures use the held-out iPhone dataset [70] of time-synchronized real multi-view captures (Section 4.4). Mixture dataset weights and other preprocessing details are in Appendices A and B.

Baselines and Metrics. We compare against video-to-video camera-conditioned baselines that share HOLO4D's task formulation: ReCamMaster [42], Gen3C [46], TrajectoryCrafter [45], EX-4D [48], and Vista4D [9][1], covering both implicit camera conditioning and explicit geometry-prior re-rendering. We follow the DAVIS-based protocol detailed in Appendix D: the first 50 DAVIS [69] clips serve as source videos, each paired with two target trajectories sampled from the trajectory pools released by ReCamMaster and Vista4D (100 source-target pairs). We report camera adherence (ATE, RPE-trans, RPE-rot), with target-camera trajectories of every generated video recovered by the same ViPE [71] pipeline, and video quality (FVD, CLIP-F / CLIP-V) following recent camera-controlled video benchmarks; novel-view RGB and downstream-reconstruction depth on real captures are reported separately in Table 2. Each baseline is run with its native preprocessing and output resolution, and all outputs are resized to a common 288×512 resolution and 49 frames before metric computation, so the reported numbers are not biased by per-method generation resolution.

[1]Vista4D is concurrent work, released roughly two weeks before the NeurIPS submission deadline; we include it for completeness rather than as a primary point of comparison.

Table 1: **Quantitative comparison.** ATE, trans. err., rot. err., and FVD are lower-is-better, while CLIP-F/V are higher-is-better. Each baseline runs at its native generation resolution (rightmost column); all outputs are resized to a common $288\times512\times49$ before metric computation. The matched-backbone comparison at 1.3B (HOLO4D-1.3B vs. ReCamMaster, both Wan2.1-T2V-1.3B) controls for backbone family; HOLO4D-14B is built on Wan2.1-I2V-14B and Vista4D on Wan2.1-T2V-14B. Best and second-best results are highlighted as best and second.

Method	ATE ↓	Trans ↓	Rot ↓	FVD ↓	CLIP-F ↑	CLIP-V ↑	Base Model	Res
ReCamMaster	0.933	0.082	0.173	652.130	0.975	0.875	`Wan2.1-T2V-1.3B`	480×832
Gen3C	0.588	0.059	0.549	794.618	0.960	0.798	`Cosmos-7B`	704×1280
HOLO4D-1.3B	0.369	0.036	0.175	843.831	0.957	0.792	`Wan2.1-T2V-1.3B`	288×512
TrajectoryCrafter	0.357	0.040	0.442	665.794	0.966	0.850	`CogVideoX-Fun-5B`	384×672
EX-4D	0.322	0.030	0.988	823.406	0.958	0.794	`Wan2.1-I2V-14B`	512×512
Vista4D	0.305	0.032	0.172	637.091	0.973	0.867	`Wan2.1-T2V-14B`	384×672
HOLO4D-14B	0.209	0.018	0.140	675.700	0.973	0.855	`Wan2.1-I2V-14B`	288×512

Implementation Details. HOLO4D is built on the 1.3B and 14B Wan 2.1 Fun-Control-Camera DiT backbones [5] (30/40 DiT blocks), with four conditioning adapters added for V2V, hybrid RGB-D, Plücker rays, and 3D tracks; all are zero- or identity-initialised so the model starts from the released VDM [72]. We use 4RC [37] as the upstream 4D reconstructor for both HOLO4D and any baseline that consumes proxy depth, point clouds, or 3D tracks (Appendix D); the pipeline is agnostic to this choice (Section 3.1). The static cache uses $K=7$ camera-coverage keyframes selected by farthest-point sampling. The DiT and adapters are fine-tuned end-to-end with $\mathcal{L}_{\mathrm{FM}}$ using AdamW [73] (lr 3×10^{-5}, batch 32) at 144×256 and 288×512 (49 frames). The 14B model trains on 32×H100 for $\sim$1 day and the 1.3B model on 2×A6000 for $\sim$7 days, and inference uses 50 denoising steps with guidance 2.5 ($\sim$1 min on H100 at 144×256). Geometry-token noise, classifier-free dropout, and the separately-trained depth-readout head used in Appendix E are detailed in Appendices A and B.

4.2 Qualitative Evaluation

Figure 3 shows qualitative comparisons across DAVIS clips with varied subjects and target trajectories sampled from the ReCamMaster and Vista4D trajectory pools (Appendix D). For each example we show the source frame, the target trajectory, and the target-view RGB synthesized by HOLO4D alongside video-to-video baselines at matched target-frame indices. HOLO4D follows the requested target trajectory and preserves source content under large viewpoint changes; the corresponding camera-adherence and video-quality numbers are reported in Table 1, and novel-view RGB and downstream-reconstruction depth on real captures are in Table 2. These examples illustrate the intended role of the geometry-aware condition: explicit 4D geometry anchors the target camera signal, while the VDM prior synthesizes target-view content in regions not directly supported by the rendered condition.

4.3 Quantitative Evaluation

Camera-control comparison. Table 1 compares HOLO4D with implicit-prior and explicit-prior video re-rendering baselines on held-out DAVIS source-target pairs. The cleanest backbone-controlled comparison is at 1.3B, where HOLO4D and ReCamMaster share the Wan2.1-T2V-1.3B backbone: HOLO4D lowers ATE from 0.933 to 0.369 and RPE-trans from 0.082 to 0.036, while remaining competitive with the 5–14B explicit-prior baselines on RPE-rot. At 14B, HOLO4D-14B (built on Wan2.1-I2V-14B) achieves the best camera adherence across the board, lowering ATE to 0.209 (vs. 0.305 for the next-best 14B baseline Vista4D, which uses Wan2.1-T2V-14B), RPE-trans to 0.018 (vs. 0.030 for EX-4D), and RPE-rot to 0.140 (vs. 0.172 for Vista4D). The matched-backbone comparison at 1.3B isolates the geometric interface, and the 14B numbers extend the gain to the larger-scale setting despite the I2V vs. T2V backbone difference, indicating that the camera-adherence improvement is driven by the geometric interface rather than by model size.

Video quality. Video-fidelity metrics (FVD, CLIP-F / CLIP-V) remain competitive but are not uniformly best. This trade-off is expected: methods with weaker camera motion stay close to input-like viewpoints and therefore score well on generic video fidelity while not actually following the requested target trajectory—ReCamMaster, for example, has the highest CLIP-F / CLIP-V but the worst ATE among Wan-1.3B systems. HOLO4D-14B is second-best on CLIP-F (0.973) and within

Table 2: **Novel-view RGB and depth on the iPhone dataset [70].** Synthesis quality is reported as masked ("m") and full-image PSNR, SSIM, and LPIPS following [9]; video depth is reported against pseudo-ground-truth depth from the upstream 4D reconstructor (4RC [2]). Since real multi-view captures provide no metric ground-truth depth, these numbers reflect target-view geometric consistency under a common reconstruction-based evaluator, not absolute metric depth. Best and second-best results are highlighted as best and second.

Method	Novel-view video synthesis						Video Depth		
	mPSNR ↑	mSSIM ↑	mLPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	AbsRel ↓	RMSE ↓	$\delta < 1.25$ ↑
ReCamMaster	10.411	0.485	0.634	10.586	0.335	0.693	0.125	0.193	0.842
Gen3C	12.422	0.466	0.489	12.198	0.307	0.550	0.108	0.161	0.898
HOLO4D-1.3B	13.605	0.485	0.486	13.213	0.330	0.563	0.093	0.157	0.902
TrajectoryCrafter	11.288	0.432	0.566	10.914	0.282	0.631	0.084	0.142	0.906
EX-4D	13.637	0.527	0.459	13.067	0.371	0.531	0.095	0.147	0.901
Vista4D	12.457	0.486	0.498	12.062	0.329	0.549	0.075	0.133	0.918
HOLO4D-14B	13.767	0.475	0.438	13.365	0.313	0.504	0.069	0.129	0.918

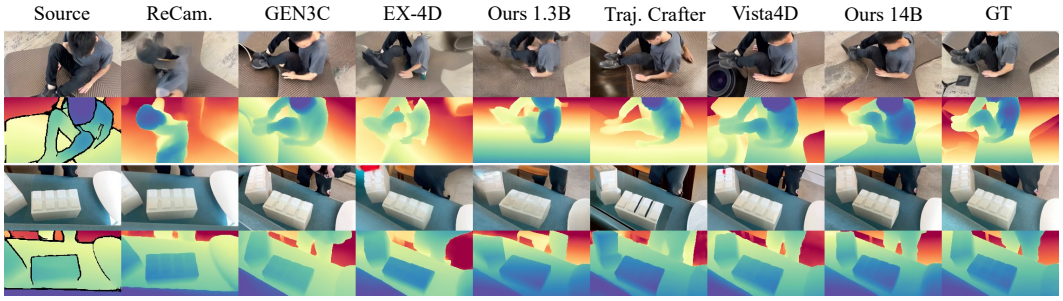


Figure 4: **Novel-view RGB and geometric consistency on the iPhone dataset.** For each example we show the source frame, the target-view RGB synthesized by HOLO4D, the predicted or reconstructed target-view depth, and side-by-side comparisons with video-to-video baselines. "GT" in the RGB panels is the held-out target-view RGB; the depth reference is pseudo-ground-truth obtained by running 4RC on the held-out camera, not metric ground-truth. HOLO4D preserves source-video content under large-baseline target trajectories; the corresponding RGB and downstream-reconstruction depth metrics are reported in Table 2.

7% of the best FVD, while leading on every camera metric, indicating that the stronger camera adherence is obtained while retaining competitive generic video quality.

4.4 Novel-view RGB and Geometric Consistency on Real Captures

We follow the Vista4D [9] protocol on the iPhone dataset [70]: each clip uses one camera as source and renders along the held-out camera trajectory, with full-image and masked PSNR / SSIM / LPIPS against the held-out RGB (mask defined as in [9]). For depth, we run the same off-the-shelf 4D reconstructor (4RC [37]) on each method's generated RGB and on the held-out camera as pseudo-ground-truth, reporting AbsRel, RMSE, and $\delta < 1.25$ accuracy.

RGB synthesis. Table 2 shows that HOLO4D-14B obtains the best PSNR (13.37 vs. 13.07 for the next-best baseline EX-4D), the best LPIPS (0.504), and the best masked PSNR / LPIPS (13.77 / 0.438); EX-4D leads SSIM and masked SSIM (0.371, 0.527). This SSIM-LPIPS trade-off mirrors the one reported by Vista4D [9]: SSIM rewards low-frequency, blur-tolerant outputs, whereas LPIPS measures perceptual similarity to the held-out target view. This is consistent with the camera-adherence results in Table 1: methods with weaker target-camera following can still obtain competitive SSIM by producing conservative, source-like views. Vista4D leads FVD and is second on CLIP-F / CLIP-V in Table 1 but is lower on the held-out target-view PSNR / LPIPS in Table 2. This highlights why generic video fidelity and target-view reconstruction accuracy should be interpreted together.

Downstream geometric consistency. On all three depth metrics, HOLO4D-14B is best (AbsRel 0.069, RMSE 0.129, $\delta < 1.25$ 0.918), with the concurrent Vista4D second. The remaining baselines, including EX-4D, fall behind on every depth metric despite being competitive on RGB SSIM. High RGB similarity alone does not guarantee that the generated video is geometrically consistent under downstream 4D reconstruction. These results indicate that HOLO4D's generated videos are not only perceptually closer to the held-out target views, but are also more compatible with downstream 4D

reconstruction: when the same off-the-shelf reconstructor is applied to every method's RGB output, ours yields substantially lower depth error. Appendix E provides additional evidence at the feature level: a lightweight head trained on the frozen backbone can decode target-view depth directly from intermediate VAE decoder features.

4.5 Ablations and Analysis

Table 3: **Ablation study.** All metrics are lower-is-better; PCD(RGB-D) is the baseline, where PCD means point cloud. single: per-frame static reprojection; multi: camera-coverage multi-frame static cache. Per-column rank is colored best, 2nd best, worst.

Ablation Setup	ATE ↓	RPE trans ↓	RPE rot ↓
Plücker Ray	0.974	0.0650	0.165
PCD(RGB)	0.385	0.0346	0.320
PCD(RGB-D)	0.393	0.0348	0.206
PCD(RGB-D) + 3D Track (single)	0.354	0.0341	0.167
PCD(RGB-D) + 3D Track (multi)	0.370	0.0309	0.163

We ablate the geometry-control design by progressively adding geometric signals at the 1.3B / 144×256 scale (Table 3).

From implicit camera control to explicit geometry. With only Plücker rays as the camera signal (Plücker Ray), adherence is poor (ATE=0.974, RPE-trans=0.0650). Adding a point-cloud RGB render (PCD(RGB), where PCD denotes point cloud) gives a $2.5\times$ and $1.9\times$ improvement on ATE / RPE-trans, and adding rendered depth and a valid mask (PCD(RGB-D)) further cuts RPE-rot from 0.320 to 0.206.

Full hybrid 4D condition. Injecting 3D tracking tokens on top of PCD(RGB-D) while keeping both static and dynamic regions rendered per-frame (PCD(RGB-D)+3D Track (single)) reaches the best ATE (0.354). Switching the static stream to the camera-coverage multi-frame cache while keeping dynamic regions per-frame—the full HOLO4D condition (PCD(RGB-D)+3D Track (multi))—further lowers the local-consistency errors to the best RPE-trans (0.0309) and RPE-rot (0.163), at a marginal cost on ATE (0.370 vs. 0.354). ATE is computed after Procrustes alignment and is dominated by long-range drift, whereas RPE-trans / RPE-rot directly measure local frame-to-frame consistency—the quantity the multi-frame static cache is designed to improve.

5 Conclusion and Future Work

We presented HOLO4D, a geometry-aware video-to-video framework for monocular novel-view 4D reconstruction that combines static fusion across camera-coverage source views, matching-frame dynamic rendering, dense 3D tracking, and target-camera rays into a stronger VDM control signal than pose-only or per-frame geometry conditioning, generating target-view RGB whose downstream 4D reconstruction is more consistent with the held-out reconstruction than that of the baselines. Appendix E adds a feature-level check: the same target-view depth is decodable directly from intermediate VAE decoder features by a frozen-backbone readout head.

HOLO4D depends on the quality of input-view geometry and 3D tracks: depth and camera errors, along with a fixed motion-magnitude threshold for the static-dynamic split, can propagate into the rendered condition. Appendix F illustrates a concrete failure mode—tracking-induced suppression of content that newly appears in later source frames. Future work could make the cache uncertainty-aware, replace the hard threshold with a learned soft mask, and decode editable 4D representations beyond a single target trajectory; more broadly, future improvements of feed-forward 3D/4D reconstruction should directly translate in better geometric control of VDMs.

Acknowledgments. This research was supported by the Sony Research Award Programme 2024 and ERC CoG UNION (101001212).

References

[1] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *CVPR*, 2025.

[2] Yihang Luo, Shangchen Zhou, Yushi Lan, Xingang Pan, and Chen Change Loy. 4RC: 4D reconstruction via conditional querying anytime and anywhere. In *ICML*, 2026.

[3] Chuhan Zhang, Guillaume Le Moing, Skanda Koppula, Ignacio Rocco, Liliane Momeni, Junyu Xie, Shuyang Sun, Rahul Sukthankar, Joëlle K. Barral, Raia Hadsell, Zoubin Ghahramani, Andrew Zisserman, Junlin Zhang, and Mehdi S. M. Sajjadi. Efficiently reconstructing dynamic scenes one d4rt at a time. In *CVPR*, 2026.

[4] Edgar Sucar, Eldar Insafutdinov, Zihang Lai, and Andrea Vedaldi. V-DPM: Video reconstruction with dynamic point maps. In *CVPR*, 2026.

[5] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wente Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[6] Jianhong Bai, Menghan Xia, Xiao Fu, Xintao Wang, Lianrui Mu, Jinwen Cao, Zuozhu Liu, Haoji Hu, Xiang Bai, Pengfei Wan, et al. Recammaster: Camera-controlled generative rendering from a single video. In *ICCV*, October 2025.

[7] Yuedong Chen, Chuanxia Zheng, Haofei Xu, Bohan Zhuang, Andrea Vedaldi, Tat-Jen Cham, and Jianfei Cai. MVSplat360: Benchmarking 360 generalizable 3D novel view synthesis from sparse views. In *NeurIPS*, 2024.

[8] Xuanchi Ren, Tianchang Shen, Jiahui Huang, Huan Ling, Yifan Lu, Merlin Nimier-David, Thomas Müller, Alexander Keller, Sanja Fidler, and Jun Gao. GEN3C: 3D-informed world-consistent video generation with precise camera control. In *CVPR*, 2025.

[9] Kuan Heng Lin, Zhizheng Liu, Pablo Salamanca, Yash Kant, Ryan Burgert, Koichi Namekata, Yiwei Zhao, Bolei Zhou, Micah Goldblum, Paul Debevec, and Ning Yu. Vista4D: Video reshooting with 4d point clouds. In *CVPR*, 2026.

[10] Mingyang Xie, Numair Khan, Tianfu Wang, Naina Dhingra, Seonghyeon Nam, Haitao Yang, Zhuo Hui, Christopher Metzler, Andrea Vedaldi, Hamed Pirsiavash, and Lei Luo. LaVR: Scene latent conditioned generative video trajectory re-rendering using large 4D reconstruction models. In *CVPR*, 2026.

[11] Edgar Sucar, Eldar Insafutdinov, Zihang Lai, and Andrea Vedaldi. V-DPM: 4D video reconstruction with dynamic point maps, 2026.

[12] Vincent Sitzmann, Semon Rezchikov, William T. Freeman, Joshua B. Tenenbaum, and Fredo Durand. Light field networks: Neural scene representations with single-evaluation rendering. In *NeurIPS*, 2021.

[13] Keunhong Park, Utkarsh Sinha, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Steven M. Seitz, and Ricardo Martin-Brualla. Nerfies: Deformable neural radiance fields. In *iccv*, 2021.

[14] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Ricardo Martin-Brualla, and Steven M. Seitz. Hypernerf: a higher-dimensional representation for topologically varying neural radiance fields. *SIGGRAPH*, 40(6), 2021.

[15] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. In *CVPR*, volume abs/2011.13961, 2021.

[16] Zhengqi Li, Qianqian Wang, Forrester Cole, Richard Tucker, and Noah Snavely. DynIBaR: Neural dynamic image-based rendering. In *CVPR*, 2023.

[17] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. Deformable 3d gaussians for high-fidelity monocular dynamic scene reconstruction. In *CVPR*, 2024.

[18] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D gaussian splatting for real-time dynamic scene rendering. In *CVPR*, 2023.

[19] Qianqian Wang, Vickie Ye, Hang Gao, Weijia Zeng, Jake Austin, Zhengqi Li, and Angjoo Kanazawa. Shape of motion: 4d reconstruction from a single video. In *ICCV*, pages 9660–9672, 2025.

[20] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. DUSt3R: Geometric 3D vision made easy. In *CVPR*, 2024.

[21] Haotong Lin, Sili Chen, Junhao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv*, 2511.10647, 2025.

[22] Qianqian Wang, Yifei Zhang, Aleksander Holynski, Alexei A Efros, and Angjoo Kanazawa. Continuous 3D perception model with persistent state. *arXiv preprint arXiv:2501.12387*, 2025.

[23] Yushi Lan, Yihang Luo, Fangzhou Hong, Shangchen Zhou, Honghua Chen, Zhaoyang Lyu, Shuai Yang, Bo Dai, Chen Change Loy, and Xingang Pan. STream3R: Scalable sequential 3D reconstruction with causal transformer. In *ICLR*, 2026.

[24] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. π^3: Permutation-equivariant visual geometry learning. In *ICLR*, 2026.

[25] Zhengqi Li, Richard Tucker, Forrester Cole, Qianqian Wang, Linyi Jin, Vickie Ye, Angjoo Kanazawa, Aleksander Holynski, and Noah Snavely. MegaSaM: accurate, fast and robust structure and motion from casual dynamic videos. In *Computer Vision and Pattern Recognition (CVPR)*, 2025.

[26] Junyi Zhang, Charles Herrmann, Junhwa Hur, Varun Jampani, Trevor Darrell, Forrester Cole, Deqing Sun, and Ming-Hsuan Yang. MonST3R: a simple approach for estimating geometry in the presence of motion. *arXiv*, 2410.03825, 2024.

[27] Nikhil Keetha, Norman Müller, Johannes Schönberger, Lorenzo Porzi, Yuchen Zhang, Tobias Fischer, Arno Knapitsch, Duncan Zauss, Ethan Weber, Nelson Antunes, Jonathon Luiten, Manuel Lopez-Antequera, Samuel Rota Bulò, Christian Richardt, Deva Ramanan, Sebastian Scherer, and Peter Kontschieder. MapAnything: universal feed-forward metric 3D reconstruction. In *3DV*, 2026.

[28] Jianyuan Wang, Minghao Chen, Shangzhan Zhang, Nikita Karaev, Johannes Schönberger, Patrick Labatut, Piotr Bojanowski, David Novotny, Andrea Vedaldi, and Christian Rupprecht. VGGT-Ω. In *CVPR*, 2026.

[29] Sili Chen, Hengkai Guo, Shengnan Zhu, Feihu Zhang, Zilong Huang, Jiashi Feng, and Bingyi Kang. Video depth anything: Consistent depth estimation for super-long videos. *arXiv:2501.12375*, 2025.

[30] Wenbo Hu, Xiangjun Gao, Xiaoyu Li, Sijie Zhao, Xiaodong Cun, Yong Zhang, Long Quan, and Ying Shan. DepthCrafter: generating consistent long depth sequences for open-world videos. In *CVPR*, 2025.

[31] Tian-Xing Xu, Xiangjun Gao, Wenbo Hu, Xiaoyu Li, Song-Hai Zhang, and Ying Shan. GeometryCrafter: consistent geometry estimation for open-world videos with diffusion priors. *arXiv*, 2504.01016, 2025.

[32] Haiwen Feng, Junyi Zhang, Qianqian Wang, Yufei Ye, Pengcheng Yu, Michael J. Black, Trevor Darrell, and Angjoo Kanazawa. St4RTrack: simultaneous 4D reconstruction and tracking in the world. In *ICCV*, 2025.

[33] Edgar Sucar, Zihang Lai, Eldar Insafutdinov, and Andrea Vedaldi. Dynamic Point Maps: A versatile representation for dynamic 3D reconstruction. In *ICCV*, 2025.

[34] Zhen Xu, Zhengqin Li, Zhao Dong, Xiaowei Zhou, Richard Newcombe, and Zhaoyang Lv. 4DGT: Learning a 4D gaussian transformer using real-world monocular videos. *arXiv*, 2506.08015, 2025.

[35] Xinhang Liu, Yuxi Xiao, Donny Y. Chen, Jiashi Feng, Yu-Wing Tai, Chi-Keung Tang, and Bingyi Kang. Trace anything: Representing any video in 4D via trajectory fields. In *ICLR*, 2026.

[36] Jay Karhade, Nikhil Varma Keetha, Tanisha Gupta, Yuchen Zhang, Akash Sharma, Sebastian Scherer, and Deva Ramanan. Any4D: Towards unified feed-forward metric 4d reconstruction. In *ICCVW*, 2025.

[37] Yihang Luo, Shangchen Zhou, Yushi Lan, Xingang Pan, and Chen Change Loy. 4RC: 4D reconstruction via conditional querying anytime and anywhere. In *ICML*, 2026.

[38] David Yifan Yao, Albert J. Zhai, and Shenlong Wang. Uni4D: unifying visual foundation models for 4D modeling from a single video. *arXiv*, 2503.21761, 2025.

[39] Zhouxia Wang, Ziyang Yuan, Xintao Wang, Yaowei Li, Tianshui Chen, Menghan Xia, Ping Luo, and Ying Shan. Motionctrl: A unified and flexible motion controller for video generation. In *SIGGRAPH*, 2024.

[40] Hao He, Yinghao Xu, Yuwei Guo, Gordon Wetzstein, Bo Dai, Hongsheng Li, and Ceyuan Yang. CameraCtrl: enabling camera control for text-to-video generation. In *ICLR*, 2025.

[41] Hao He, Ceyuan Yang, Shanchuan Lin, Yinghao Xu, Meng Wei, Liangke Gui, Qi Zhao, Gordon Wetzstein, Lu Jiang, and Hongsheng Li. CameraCtrl II: Dynamic scene exploration via camera-controlled video diffusion models. In *ICCV*, 2025.

[42] Jianhong Bai, Menghan Xia, Xiao Fu, Xintao Wang, Lianrui Mu, Jinwen Cao, Zuozhu Liu, Haoji Hu, Xiang Bai, Pengfei Wan, and Di Zhang. ReCamMaster: camera-controlled generative rendering from a single video. In *ICCV*, 2025.

[43] Basile Van Hoorick, Rundi Wu, Ege Ozguroglu, Kyle Sargent, Ruoshi Liu, Pavel Tokmakov, Achal Dave, Changxi Zheng, and Carl Vondrick. Generative camera dolly: Extreme monocular dynamic novel view synthesis. In *ECCV*, 2024.

[44] Sherwin Bahmani, Ivan Skorokhodov, Aliaksandr Siarohin, Willi Menapace, Guocheng Qian, Michael Vasilkovsky, Hsin-Ying Lee, Chaoyang Wang, Jiaxu Zou, Andrea Tagliasacchi, David B. Lindell, and Sergey Tulyakov. VD3D: taming large video diffusion transformers for 3D camera control. *arXiv*, 2024.

[45] Mark Yu, Wenbo Hu, Jinbo Xing, and Ying Shan. TrajectoryCrafter: redirecting camera trajectory for monocular videos via diffusion models. In *ICCV*, 2025.

[46] Xuanchi Ren, Tianchang Shen, Jiahui Huang, Huan Ling, Yifan Lu, Merlin Nimier-David, Thomas Müller, Alexander Keller, Sanja Fidler, and Jun Gao. GEN3C: 3D-informed world-consistent video generation with precise camera control. *arXiv*, 2503.03751, 2025.

[47] Meng You, Zhiyu Zhu, Hui Liu, and Junhui Hou. NVS-Solver: video diffusion model as zero-shot novel view synthesizer. In *ICLR*, 2025.

[48] Tao Hu, Haoyang Peng, Xiao Liu, and Yuewen Ma. EX-4D: EXtreme viewpoint 4D video synthesis via depth watertight mesh. *arXiv*, 2506.05554, 2025.

[49] Kuan Heng Lin, Zhizheng Liu, Pablo Salamanca, Yash Kant, Ryan Burgert, Koichi Namekata, Yiwei Zhao, Bolei Zhou, Micah Goldblum, Paul Debevec, and Ning Yu. Vista4D: Video reshooting with 4D point clouds. In *CVPR*, 2026.

[50] Koichi Namekata, Yash Kant, Zhizheng Liu, Ryan D. Burgert, Yuancheng Xu, Kuan Heng Lin, Emmett Steven, Julien Philip, Li Ma, Andrea Vedaldi, Paul Debevec, and Ning Yu. Go-with-the-track: Video compositing and motion control with point tracking. In *SIGGRAPH Conference Papers '26, July 19–23, 2026, Los Angeles, CA, USA*, SIGGRAPH Conference Papers '26, New York, NY, USA, 2026. Association for Computing Machinery.

[51] Mingyang Xie, Numair Khan, Tianfu Wang, Naina Dhingra, Seonghyeon Nam, Haitao Yang, Zhuo Hui, Christopher Metzler, Andrea Vedaldi, Hamed Pirsiavash, and Lei Luo. LaVR: Latent space conditioned video re-rendering using large 4D reconstruction models. In *CVPR*, 2026.

[52] Bingxin Ke, Anton Obukhov, Shengyu Huang, Nando Metzger, Rodrigo Caye Daudt, and Konrad Schindler. Repurposing diffusion-based image generators for monocular depth estimation. In *CVPR*, 2024.

[53] Gangwei Xu, Haotong Lin, Hongcheng Luo, Xianqi Wang, Jingfeng Yao, Lianghui Zhu, Yuechuan Pu, Cheng Chi, Haiyang Sun, Bing Wang, et al. Pixel-perfect depth with semantics-prompted diffusion transformers. *arXiv preprint arXiv:2510.07316*, 2025.

[54] Zeren Jiang, Chuanxia Zheng, Iro Laina, Diane Larlus, and Andrea Vedaldi. Geo4D: Leveraging video generators for geometric 4D scene reconstruction. In *ICCV*, 2025.

[55] Zhaoxi Chen, Tianqi Liu, Long Zhuo, Jiawei Ren, Zeng Tao, He Zhu, Fangzhou Hong, Liang Pan, and Ziwei Liu. 4DNeX: feed-forward 4D generative modeling made easy. *arXiv*, 2025.

[56] Hyojun Go, Dominik Narnhofer, Goutam Bhat, Prune Truong, Federico Tombari, and Konrad Schindler. Text-to-3d by stitching a multi-view reconstruction network to a video generator. In *ICLR*, 2026.

[57] Jiaxin Huang, Yuanbo Yang, Bangbang Yang, Lin Ma, Yuewen Ma, and Yiyi Liao. Gen3R: 3D scene generation meets feed-forward reconstruction. In *CVPR*, 2026.

[58] Ruijie Zhu, Jiahao Lu, Wenbo Hu, Xiaoguang Han, Jianfei Cai, Ying Shan, and Chuanxia Zheng. Motioncrafter: Dense geometry and motion reconstruction with a 4d vae. In *CVPR*, 2026.

[59] Zhaoxi Chen, Tianqi Liu, Long Zhuo, Jiawei Ren, Zeng Tao, He Zhu, Fangzhou Hong, Liang Pan, and Ziwei Liu. 4dnex: Feed-forward 4d generative modeling made easy. *arXiv preprint arXiv:2508.13154*, 2025.

[60] Xingchao Liu, Chengyue Gong, and qiang liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *ICLR*, 2023.

[61] Zeren Jiang, Yushi Lan, Yihang Luo, Yufan Deng, Zihang Lai, Edgar Sucar, Christian Rupprecht, Iro Laina, Diane Larlus, Chuanxia Zheng, and Andrea Vedaldi. Syn4D: A multiview synthetic 4D dataset. In *ECCV*, 2026.

[62] Klaus Greff, Francois Belletti, Lucas Beyer, Carl Doersch, Yilun Du, Daniel Duckworth, David J Fleet, Dan Gnanapragasam, Florian Golemo, Charles Herrmann, Thomas Kipf, Abhijit Kundu, Dmitry Lagun, Issam Laradji, Hsueh-Ti (Derek) Liu, Henning Meyer, Yishu Miao, Derek Nowrouzezahrai, Cengiz Oztireli, Etienne Pot, Noha Radwan, Daniel Rebain, Sara Sabour, Mehdi S. M. Sajjadi, Matan Sela, Vincent Sitzmann, Austin Stone, Deqing Sun, Suhani Vora, Ziyu Wang, Tianhao Wu, Kwang Moo Yi, Fangcheng Zhong, and Andrea Tagliasacchi. Kubric: a scalable dataset generator. In *CVPR*, 2022.

[63] Jianhong Bai, Menghan Xia, Xintao Wang, Ziyang Yuan, Xiao Fu, Zuozhu Liu, Haoji Hu, Pengfei Wan, and Di Zhang. Syncammaster: Synchronizing multi-camera video generation from diverse viewpoints. *arXiv preprint arXiv:2412.07760*, 2024.

[64] Lu Ling, Yichen Sheng, Zhi Tu, Wentian Zhao, Cheng Xin, Kun Wan, Lantao Yu, Qianyu Guo, Zixun Yu, Yawen Lu, et al. DL3DV-10k: A large-scale scene dataset for deep learning-based 3D vision. In *CVPR*, 2024.

[65] Tinghui Zhou, Richard Tucker, John Flynn, Graham Fyffe, and Noah Snavely. Stereo magnification: Learning view synthesis using multiplane images. *arXiv*, 2018.

[66] Po-Han Huang, Kevin Matzen, Johannes Kopf, Narendra Ahuja, and Jia-Bin Huang. DeepMVS: Learning multi-view stereopsis. In *CVPR*, 2018.

[67] Wenshan Wang, Delong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. TartanAir: A dataset to push the limits of visual SLAM. In *IROS*, 2020.

[68] Haotong Lin, Sili Chen, Jun Hao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.

[69] Jordi Pont-Tuset, Federico Perazzi, Sergi Caelles, Pablo Arbeláez, Alexander Sorkine-Hornung, and Luc Van Gool. The 2017 davis challenge on video object segmentation. *arXiv:1704.00675*, 2017.

[70] Hang Gao, Ruilong Li, Shubham Tulsiani, Bryan Russell, and Angjoo Kanazawa. Monocular dynamic view synthesis: A reality check. In *NeurIPS*, 2022.

[71] Jiahui Huang, Qunjie Zhou, Hesam Rabeti, Aleksandr Korovko, Huan Ling, Xuanchi Ren, Tianchang Shen, Jun Gao, Dmitry Slepichev, Chen-Hsuan Lin, Jiawei Ren, Kevin Xie, Joydeep Biswas, Laura Leal-Taixe, and Sanja Fidler. ViPE: video pose engine for 3D geometric perception. *arXiv*, 2508.10934, 2025.

[72] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *ICCV*, 2023.

[73] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[74] Hyung Won Chung, Noah Constant, Xavier Garcia, Adam Roberts, Yi Tay, Sharan Narang, and Orhan Firat. Unimax: Fairer and more effective language sampling for large-scale multilingual pretraining. *arXiv preprint arXiv:2304.09151*, 2023.

[75] Sherwin Bahmani, Tianchang Shen, Jiawei Ren, Jiahui Huang, Yifeng Jiang, Haithem Turki, Andrea Tagliasacchi, David B. Lindell, Zan Gojcic, Sanja Fidler, Huan Ling, Jun Gao, and Xuanchi Ren. Lyra: Generative 3D scene reconstruction via video diffusion model self-distillation. In *ICLR*, 2026.

[76] Tianchang Shen, Sherwin Bahmani, Kai He, Sangeetha Grama Srinivasan, Tianshi Cao, Jiawei Ren, Ruilong Li, Zian Wang, Nicholas Sharp, Zan Gojcic, Sanja Fidler, Jiahui Huang, Huan Ling, Jun Gao, and Xuanchi Ren. Lyra 2.0: Explorable generative 3d worlds. *arXiv preprint arXiv:2604.13036*, 2026.

[77] Ruijie Zhu, Jiahao Lu, Wenbo Hu, Xiaoguang Han, Jianfei Cai, Ying Shan, and Chuanxia Zheng. Motioncrafter: Dense geometry and motion reconstruction with a 4d vae. In *CVPR*, 2026.

[78] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. Pytorch fsdp: Experiences on scaling fully sharded data parallel, 2023.

[79] Chaoyang Wang, Ashkan Mirzaei, Vidit Goel, Willi Menapace, Aliaksandr Siarohin, Michael Vasilkovsky, Ivan Skorokhodov, Vladislav Shakhrai, Sergei Korolev, Sergey Tulyakov, and Peter Wonka. Fused view-time attention and feedforward reconstruction for 4d scene generation. In *NeurIPS*, 2026.

[80] Zeren Jiang, Yushi Lan, Yihang Luo, Yufan Deng, Zihang Lai, Edgar Sucar, Christian Rupprecht, Iro Laina, Diane Larlus, Chuanxia Zheng, and Andrea Vedaldi. Syn4D: A multiview synthetic 4D dataset, 2026.

[81] Jiahui Huang, Qunjie Zhou, Hesam Rabeti, Aleksandr Korovko, Huan Ling, Xuanchi Ren, Tianchang Shen, Jun Gao, Dmitry Slepichev, Chen-Hsuan Lin, Jiawei Ren, Kevin Xie, Joydeep Biswas, Laura Leal-Taixe, and Sanja Fidler. Vipe: Video pose engine for 3d geometric perception. In *NVIDIA Research Whitepapers arXiv:2508.10934*, 2025.

A Implementation Details

Backbone. HOLO4D is built on Wan 2.1 Fun-Control-Camera [5], a DiT-based video diffusion backbone with a 16-channel video VAE ($\times 8$ spatial and $\times 4$ temporal compression), a umT5-XXL text encoder [74], and 30 / 40 full-attention DiT blocks at the 1.3B / 14B parameter scales. We fine-tune the entire DiT end-to-end rather than using LoRA adapters. The newly introduced conditioning modules are the source-target V2V domain embedding, the hybrid RGB-D/valid-mask encoder, the Plücker-ray camera adapter with per-block camera projectors, and the 3D-track token injector. All condition adapters are zero- or identity-initialized where applicable, so the model starts from the released Wan checkpoint and gradually learns to use the additional geometry-control signals [72]. The depth-readout head used for the feature-level analysis (Appendix E) is trained as a separate stage on top of the frozen backbone and is not part of the conditioning stack.

Hybrid geometry cache. For each 49-frame source clip, we build the static cache from $K = 7$ camera-coverage keyframes. For each source camera, we construct a descriptor by concatenating clip-normalized camera translation with 0.5 times the normalized camera viewing direction. We always include the first and last frames, then select the remaining keyframes by farthest-point sampling in this descriptor space. The selected indices are sorted temporally before building the static point cloud. Dynamic pixels are detected by thresholding 3D-track motion (we use 4RC [2] as the upstream 4D reconstructor in our experiments, but other choices such as VDPM [11] or D4RT [3] can be substituted) with threshold τ_{dyn}. Pixels above the threshold are excluded from the static multi-frame cache and are instead rendered with the time-synchronized per-frame fallback. The static and dynamic renders are combined by per-pixel target-view z-buffering. Concretely, for each target pixel u we define a static-stream weight

$$w_j^{\text{st}}(u) = \mathbf{1}\big[\bar{O}^{\text{t}}_{j,\text{st}}(u){=}1 \ \wedge \ \big(\bar{O}^{\text{t}}_{j,\text{dyn}}(u){=}0 \ \vee \ \bar{D}^{\text{t}}_{j,\text{st}}(u) \le \bar{D}^{\text{t}}_{j,\text{dyn}}(u)\big)\big], \tag{5}$$

$$w_j^{\text{dyn}}(u) = \bar{O}^{\text{t}}_{j,\text{dyn}}(u)\,\big(1 - w_j^{\text{st}}(u)\big), \tag{6}$$

i.e., the static stream wins where it is valid and the dynamic stream is either invalid or farther; the dynamic stream wins everywhere else it is valid. The merged render is then

$$\bar{X}^{\text{t}}_j = w_j^{\text{st}} \odot \bar{X}^{\text{t}}_{j,\text{st}} + w_j^{\text{dyn}} \odot \bar{X}^{\text{t}}_{j,\text{dyn}} \quad \text{for} \quad X \in \{I, D\}, \qquad \bar{O}^{\text{t}}_j = \bar{O}^{\text{t}}_{j,\text{st}} \vee \bar{O}^{\text{t}}_{j,\text{dyn}}, \tag{7}$$

which produces the rendered RGB, masked and normalized target-camera z-depth (following Geo4D [54]), and valid mask used in Equation (2). Rendering uses a Triton-accelerated `softsplat` operator with depth-weighted visibility blending, as in Gen3C [8] and Lyra [75, 76].

Condition encoders. The hybrid RGB-D condition tensor $\mathbf{C}^{\text{t}}_j$ defined in Equation (2) is encoded by a dedicated control patch-embedding (a Conv3d) and added to the patchified video tokens at the DiT input; regions without valid reprojection support are zero-padded and flagged by the valid-mask channel. During training, the dense geometry tokens receive continuous Gaussian noise with level sampled from $\mathcal{U}(0, 0.2)$ to improve robustness to imperfect source depth, splatting holes, and static/dynamic mask errors, following geometry-conditioning noise strategies [77, 9]. Classifier-free-style conditioning dropout with probability 0.1 is applied independently to the hybrid RGB-D branch and the 3D-track branch.

Camera and track injection. Target-camera extrinsics and intrinsics are converted into per-pixel Plücker ray maps in the canonical source frame. A lightweight camera adapter maps these ray maps to DiT-resolution tokens, which are injected into each DiT block through zero-initialised camera projections and identity-initialised output projectors. For the 3D tracking branch, source-view 3D tracks are encoded with sinusoidal positional encoding followed by an MLP, scattered into the target-view footprint, and patchified by a Conv3d stack. The resulting track tokens are injected at every DiT block by a learned concat-project head that concatenates them with the DiT hidden state and projects back to the model width, with the hidden-state portion identity-initialised and the track portion zero-initialised. The hybrid RGB-D condition enters separately at the patch-embedding input through a dedicated control patch-embedding, rather than being fused with the track tokens.

Optimization and compute. Optimization uses AdamW [73] with a constant learning rate of 3×10^{-5}, $\beta = (0.9, 0.95)$, and no warmup. We sample flow-matching timesteps $t \sim \mathcal{U}(0, 1000)$ with shift $\sigma_{\text{shift}} = 5$. The effective batch size is 32 videos, reached with per-rank micro-batch size 1 and gradient accumulation. The 14B model is trained in bf16 with FSDP full sharding [78] on 32×H100 GPUs for approximately 1 day at $144{\times}256$ and $288{\times}512$ video resolutions with 49 frames.

The 1.3B variant is trained with DDP on 2×A6000 GPUs for approximately 7 days under the same video configurations. Gradient checkpointing is enabled throughout to fit the full conditioning stack. At inference, we use 50 denoising steps, $\sigma_{\text{shift}} = 5$, and guidance scale 2.5; a $144\times256\times49$ sample takes approximately 1 minute on a single H100, while a $288\times512\times49$ sample takes approximately 4 minutes.

B Training Data

Table 4: **Training dataset mix.** Sampling is the per-step proportion of the training mixture (232.5K steps per epoch). "Conditioning" indicates which signals are supervised: "Full" uses RGB, depth, 3D tracks, and Plücker rays, while "Camera only" supervises only Plücker rays because depth and 3D-track annotations are not reliable on those datasets. Real-world static videos are re-annotated with DepthAnything 3 [68]; synthetic and Kubric splits use ground-truth depth. Static scenes are processed into overlapping (source, target) video chunks following [79].

#	Dataset	Scene Type	Real / Synth.	Dyn. / Static	Conditioning	Sampling (%)
1	Syn4D [80]	Mixed	Synthetic	Dynamic	Full (GT)	32.3
2	DL3DV [64]	Mixed	Real	Static	Full (DA3)	22.6
3	MultiCamVideo [42]	Mixed	Synthetic	Dynamic	Camera only	10.8
4	Kubric [62]	Object	Synthetic	Dynamic	Full (GT)	10.8
5	TartanAir [67]	Mixed	Synthetic	Static	Full (GT)	10.8
6	MVS-Synth [66]	Outdoor	Synthetic	Static	Full (GT)	5.4
7	SynCamVideo [63]	Mixed	Synthetic	Dynamic	Camera only	4.3
8	RE10K [65]	Mixed	Real	Static	Full (DA3)	3.2

HOLO4D is trained on a mixture of dynamic and static datasets summarized in Table 4.

Static-scene processing. For static scenes (DL3DV [64], RE10K [65], MVS-Synth [66], TartanAir [67]), we follow the chunking scheme of 4Real-Video-V2 [79]: each video is split into temporally overlapping chunks from which (source, target) pairs are sampled, so that the two clips share visual content while differing in viewpoint. Real-world videos (DL3DV, RE10K) are re-annotated with DepthAnything 3 [68] to provide dense depth; synthetic videos (MVS-Synth, TartanAir) use their native ground-truth depth.

Dynamic-scene data. The largest single source is Syn4D [80], a multi-view synthetic 4D dataset with ground-truth depth, dense 3D tracks, and multi-camera trajectories that supervises every conditioning branch. Kubric [62], which we render in-house with larger camera motion following the setup of VDPM [11] (both 49- and 81-frame splits, the latter strided down to 49 frames), supplies the same full-label supervision on object-centric scenes. The multi-camera datasets MultiCamVideo [42] and SynCamVideo [63] are used in camera-only mode—only Plücker rays are supplied as conditioning—because their depth and 3D-track annotations are not reliable enough to supervise the geometry channels.

Sampling. Per-step sampling weights are listed in Table 4. Dynamic scenes account for $\sim58\%$ of training samples and static scenes for the remaining $\sim42\%$, with Syn4D alone contributing roughly one third of all training steps to expose the model to a wide range of dynamic 4D scenes with reliable 3D labels.

C More qualitative results on in-the-wild videos

As shown in Figure 5, our HOLO4D generalizes well across many dynamic scenes, including a variety of social activities.

D Camera-Adherence Evaluation on DAVIS

This section details the protocol used for the camera-adherence and video-quality comparison in Table 1.

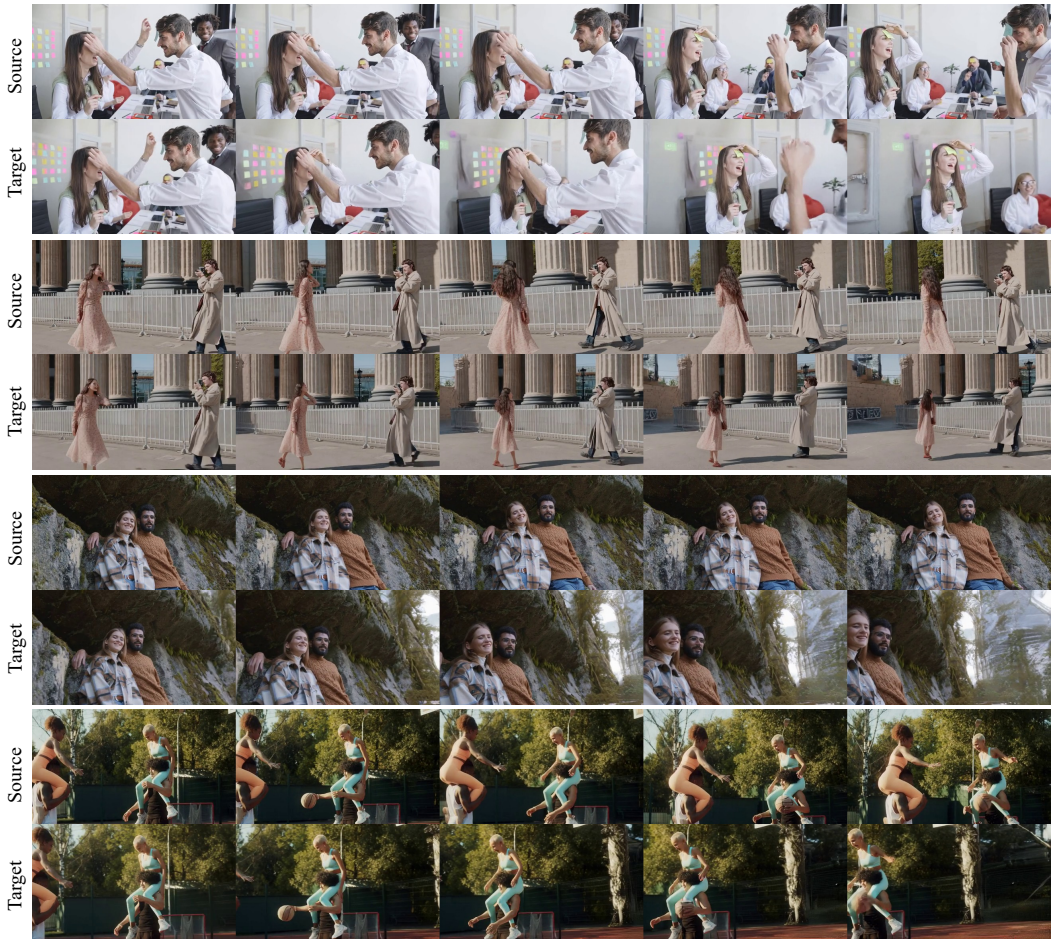


Figure 5: **Qualitative results on real-world videos.** For each example we show the source frame and the target-view RGB synthesized by HOLO4D side-by-side. HOLO4D follows the requested target trajectory and preserves source content under large viewpoint changes.

Source videos and target trajectories. We evaluate on the first 50 videos of DAVIS [69], treating each clip as a source video. For each clip we randomly sample two target camera trajectories from the trajectory pools released with ReCamMaster [42] and Vista4D [9], giving 100 source-target pairs in total; the same pairs are used by every baseline.

Baseline implementations. All baselines are run from their official open-sourced code and released checkpoints; we do not retrain or fine-tune them. Different baselines have different preprocessing requirements (for example, Vista4D additionally segments dynamic objects with SAM), and we follow each method's native preprocessing pipeline so that every baseline runs in its intended configuration. To remove confounding from upstream geometry differences, every method that consumes proxy depth, point clouds, or 3D tracks is fed the same input-view geometry estimated by 4RC [2] on the source video, matching the upstream reconstructor used by HOLO4D.

Resolution and metrics. Each method generates novel-view RGB at its officially supported resolution. Before metric computation, all generated videos and ground-truth target views are resized to a common $288\times512\times49$, so the reported numbers are not biased by per-method generation resolution. Camera-adherence metrics (ATE, RPE-trans, RPE-rot) are computed by recovering the camera trajectory of each generated video with ViPE [81] and comparing against the sampled ground-truth target trajectory; the same trajectory-recovery pipeline is applied to every method's output for a like-for-like comparison.

E Target-view Depth Readout

The depth readout is not part of the method pipeline—the RGB diffusion backbone alone produces the target-view video. We include it here as a feature-level analysis showing that the intermediate decoder features of a geometry-conditioned VDM already carry target-view geometry: a small head can read it out without modifying the backbone or running a second monocular depth estimator on the generated RGB. Concretely, at a fixed intermediate denoising step $m^\star$, we pass the current target RGB latent through the Wan VAE decoder and attach a lightweight head h_ψ to decoder feature $F_\ell^{(m^\star)}$:

$$\hat{\mathcal{Z}}^{\mathrm{t}} = h_\psi\left(F_\ell^{(m^\star)}\right), \qquad \hat{\mathcal{D}}^{\mathrm{t}} = \exp(\hat{\mathcal{Z}}^{\mathrm{t}}) - \epsilon, \tag{8}$$

where $\hat{\mathcal{Z}}^{\mathrm{t}}$ is target-view log-depth and ϵ is a small constant for numerical stability. The head is trained as a lightweight second stage on top of a frozen VDM backbone: the RGB diffusion model is first trained to convergence with $\mathcal{L}_{\mathrm{FM}}$ alone (Equation (4)), and only h_ψ is then optimized while the DiT, condition adapters, and VAE remain frozen. We supervise log-depth where target-view depth is available:

$$\mathcal{L}_{\mathrm{depth}} = \frac{1}{\|M^{\mathrm{t}}\|_1} \left\| M^{\mathrm{t}} \odot \left(\hat{\mathcal{Z}}^{\mathrm{t}} - \log(\mathcal{D}^{\mathrm{t}} + \epsilon)\right) \right\|_1 . \tag{9}$$

Because the backbone is frozen during this stage, the readout head cannot perturb the RGB denoising trajectory, and the predicted target-view log-depth is, by construction, a function of the geometry-aware features the VDM already produces.

Table 5: **Depth-head readout ablation.** We probe geometry-aware features in frozen VDMs trained with two camera-conditioning regimes and read out target-view depth via the same lightweight head h_ψ. Results are reported on the held-out MVS-Synth split at the same depth-head training budget (N=5 000 optimizer steps); lower is better for AbsRel, higher is better for $\delta_{1.25}$.

Camera condition	AbsRel ↓	$\delta_{1.25}$ ↑
Plücker rays only	0.273	0.530
Plücker + multi-frame RGB-D + 3D tracks (ours)	0.255	0.554

Effect of camera conditioning on geometry readout. We use the depth readout as a feature-level probe to ask whether richer camera conditioning makes the VDM features more geometry-aware. We compare two frozen VDM backbones trained with the same diffusion objective but different camera-conditioning regimes: a Plücker-rays-only baseline, and the full multi-frame soft-splat RGB-D plus persistent 3D tracks conditioning used by HOLO4D. For each backbone we train an identical depth head h_ψ on the same MVS-Synth split (a single held-out scene) for the same number of optimizer steps, with all other hyper-parameters fixed. Depth-readout quality after $5{,}000$ optimizer steps is reported in Table 5: the Plücker + RGBD + 3D-tracks conditioning yields lower AbsRel and higher $\delta_{1.25}$ than Plücker rays alone, indicating that the richer conditioning produces intermediate decoder features from which target-view depth is more directly readable. Both backbones, however, allow the head to recover meaningful target-view depth, supporting the broader claim that camera-conditioned VDM features encode target-view geometry that does not need to be re-estimated from RGB.

F Limitations and Failure Cases

HOLO4D's output quality is bottlenecked by the upstream 3D tracker. Feed-forward 3D-track estimators anchor their tracks on the first frame of the input video and propagate forward, so content that is occluded at frame 0 and only becomes visible later in the source video carries no 3D-track signal during conditioning. Because the model is trained to follow the explicit geometric scaffold, it tends to under-generate or omit these unanchored regions in the target view—a behavior we call *tracking-induced content suppression*.

Figure 6 illustrates this failure on a DAVIS clip: regions newly entering the frame in the second half of the source video lack track support, and the corresponding target frames either omit them or hallucinate stale content. Stronger 3D trackers with multi-anchor initialization, tracker-uncertainty exposed to the conditioning, or a soft fallback to image-based completion in track-empty regions are natural directions to mitigate this failure.



Figure 6: **Failure case: tracking-induced content suppression.** First row: 8 uniformly-sampled frames of the source video. Bottom rows: HOLO4D's target-view RGB along two zoom-in target trajectories. Red boxes mark regions where objects that newly enter later source frames are missing from the generated target video, because they lack 3D-track conditioning anchored on frame 0.

G Broader Impacts

HOLO4D enables controllable novel-view video synthesis from a single source video, with potential positive uses in content creation, education, immersive media, robotics simulation, and large-scale 4D dataset construction for training perception models. Like other camera-controlled video diffusion systems, HOLO4D inherits the dual-use risks of its pretrained backbone (Wan 2.1): the same generative prior that completes disoccluded regions can in principle be misused to fabricate misleading footage of real scenes. Three properties of our system mitigate, but do not eliminate, this risk: *(i)* every output frame is conditioned on geometry recovered from a real source video, so unconditional "hallucinated" scenes cannot be produced from text alone; *(ii)* the explicit valid mask makes the support of source observations directly inspectable, which is useful for downstream forensic analysis; and *(iii)* we will follow the release license and safety guidelines of the Wan 2.1 base model when distributing checkpoints. We do not introduce a new pretrained foundation model; risks beyond those already present in the publicly released Wan 2.1 backbone are not increased by this work.

H Existing Assets and Licenses

All datasets and pretrained components used in this work are publicly available under research-permissive licenses and are used in accordance with their original terms of use. Training data are drawn from Syn4D [80], Kubric [62], MultiCamVideo [42], SynCamVideo [63], DL3DV [64], RE10K [65], MVS-Synth [66], and TartanAir [67]; quantitative camera-adherence evaluation uses DAVIS [69] and novel-view RGB / depth evaluation uses the held-out iPhone dataset [70]. Pretrained components include the Wan 2.1 Fun-Control-Camera backbone [5], the umT5-XXL text encoder [74], DepthAnything 3 [68], and 4RC [2]. Each asset is cited at first use in the main paper, and all are used solely for non-commercial academic research as permitted by their respective licenses.